UNIVERSITATEA TEHNICĂ A MOLDOVEI
FACULTATEA CALCULATOARE, INFORMATICĂ ȘI MICROELECTRONICĂ
DEPARTAMENTUL INGINERIA SOFTWARE ȘI AUTOMATICĂ

Lucrare de laborator Nr. 1

Criptografie

Cifrul Cezar

Realizat: **Crupa Gheorghe, st. gr. SI-266**

Verificat: **Maia ZAICA, asist. univ.**

Chișinău, 2026

1 SARCINA LUCRĂRII DE LABORATOR

1.1 Cifrul Cezar

2 Definirea problemei

Pentru a implementa cifrul Cezar pentru alfabetul englez într-unul din limbajele de programare. Se va folosi doar codificarea literelor conform Tabelului 1 (nu se vor folosi codificările specificate de limbajul de programare, de ex. ASCII sau Unicode). Valorile cheii vor fi între 1 și 25 inclusiv, nicio altă valoare nefiind permisă. Valorile caracterelor din text sunt între „A” și „Z”, „a” și „z”, nicio altă valoare nefiind permisă. Dacă utilizatorul introduce alte valori, va fi sugerat intervalul corect. Înainte de criptare, textul va fi convertit în majuscule, iar spațiile vor fi eliminate. Utilizatorul va putea alege operația – criptare sau decriptare – și va putea introduce cheia, mesajul sau textul cifrat, obținând mesajul cifrat, respectiv mesajul decriptat.

2.1 Cifrul Cezar cu 2 chei

Definirea problemei

Să se implementeze algoritmul cifrului Cezar cu 2 chei, păstrând condițiile exprimate în Sarcina 1. În plus, cheia 2 trebuie să fie formată doar din litere ale alfabetului latin și să aibă o lungime de cel puțin 7.

3 DESFĂȘURAREA LUCRĂRII

3.1 Teorie

În criptografie, cifrul Cezar, cunoscut și ca cifrul lui Cezar, cifrul de deplasare, codul lui Cezar sau deplasarea Cezar, este una dintre cele mai simple și mai cunoscute tehnici de criptare. Este un tip de cifru prin substituție în care fiecare literă din textul clar este înlocuită cu o literă aflată la un număr fix de poziții mai departe în alfabet. De exemplu, pentru o deplasare la stânga de 3, D ar fi înlocuit cu A, E ar deveni B și așa mai departe. Metoda poartă numele lui Iulius Cezar, care a folosit-o în corespondența sa privată.

Etapa de criptare realizată de cifrul Cezar este adesea încorporată ca parte a unor scheme mai complexe, precum cifrul Vigenère, și mai are aplicații moderne în sistemul ROT13. Ca și celelalte cifruri prin substituție cu un singur alfabet, cifrul Cezar este ușor de spart și nu oferă nicio securitate a comunicațiilor în practica modernă.

Cheia secretă k, care este aceeași pentru criptare și decriptare, constă dintr-un număr care indică deplasarea alfabetică, adică $k\epsilon\{1,2,3,\ldots,n-1\}$, unde n este lungimea alfabetului. Criptarea și decriptarea mesajului cu cifrul Cezar pot fi definite prin următoarele formule:

$$c = e_k(x) = x + k(mod n)$$

$$m = d_k(x) = y - k(mod n)$$

unde x și y reprezintă reprezentarea numerică a caracterelor respective din textul clar m și din textul cifrat c. Funcția numită Modulo ($amodb$) returnează restul împărțirii întregi a numărului întreg a la numărul întreg b.

Această metodă de criptare poartă numele lui $JuliusCaesar$, care a folosit-o pentru a comunica cu generalii săi, utilizând cheia $k = 3$ (Tabelul 1). De exemplu, pentru $k = 3$, avem:

$$e_k(S) = 18 + 3(mod 26) = 21 = V$$

$$d_k(V) = 21 - 3(mod 26) = 18 = S$$

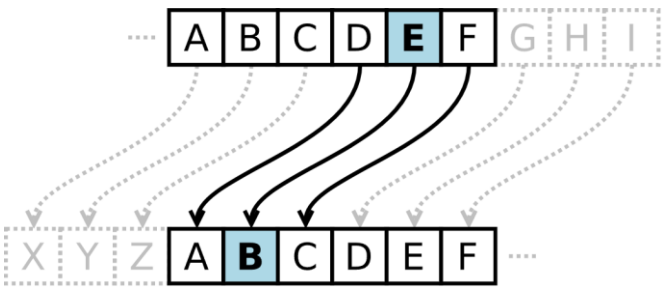


Figura 1. *Exemplu de deplasare alfabetică*

Acțiunea unui cifru Cezar constă în înlocuirea fiecărei litere din textul clar cu o altă literă, aflată la un număr fix de poziții mai departe în alfabet. Cifrul ilustrat aici folosește o deplasare la stânga de trei poziții, astfel încât (de exemplu) fiecare apariție a literei E din textul clar devine B în textul cifrat.

Tabelul 1. Cifrul Cezar cu cheia $k = 3$

0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**	**16**	**17**	**18**	**19**	**20**	**21**	**22**	**23**	**24**	**25**	**27**	**28**	**29**	**30**	**31**
A	Ă	Â	B	C	D	E	F	G	H	I	Î	J	K	L	M	N	O	P	Q	R	S	Ș	T	Ț	U	V	W	X	Y	Z

Pentru a spori rezistența criptografică a cifrului Cezar, se poate aplica o permutare a alfabetului utilizând un cuvânt-cheie (a nu se confunda cu cheia de bază a cifrului). Această cheie poate fi orice succesiune de litere din alfabet, fie că este un cuvânt din vocabular, fie unul fără sens.

Fie a doua cheie $k_2 = SECURITATE$

Aplicăm această cheie asupra alfabetului:

$$A\,Ă\,Â\,B\,C\,D\,E\,F\,G\,H\,I\,Î\,J\,K\,L\,M\,N\,O\,P\,Q\,R\,S\,Ș\,T\,Ț\,U\,V\,W\,X\,Y\,Z$$

și obținem:

$$S\,E\,C\,U\,R\,I\,T\,A\,A\,B\,Â\,D\,F\,G\,H\,Î\,J\,K\,L\,M\,N\,O\,P\,Q\,Ș\,Ț\,V\,W\,X\,Y\,Z$$

Această nouă ordine se obține plasând literele lui k_2 la început, urmate de celelalte litere ale alfabetului în ordinea lor naturală. Este important de menționat că literele nu se repetă; cu alte cuvinte, dacă o literă apare de mai multe ori, ea este plasată o singură dată.

3.2 Implementare

Figura 1 afișează ceea ce vede utilizatorul la rularea programului.

```
=== CIFRUL CEZAR (1 CHEIE SI 2 CHEI) ===
1. Single-Key Caesar Cipher
2. Two-Key Caesar Cipher
Alege optiunea (1 sau 2):
```

Figura 1. Ce obținem la rularea programului nostru

Există 2 opțiuni pe care utilizatorul le poate alege. Cifrul Cezar cu o cheie, Cifrul Cezar cu două chei și ieșirea din program.

Figura 2. Criptare și decriptare folosind o singură cheie (k1 = 5):

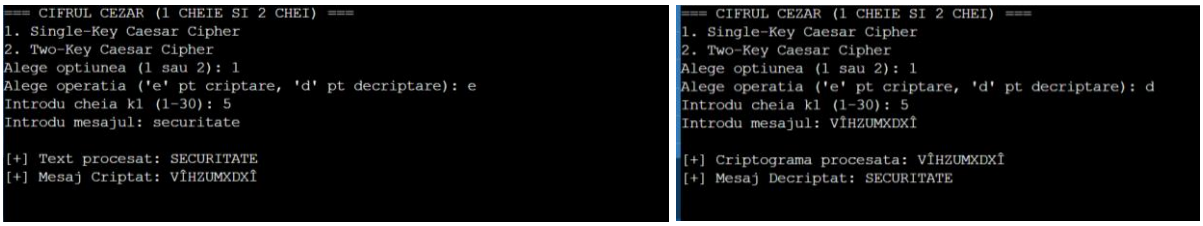
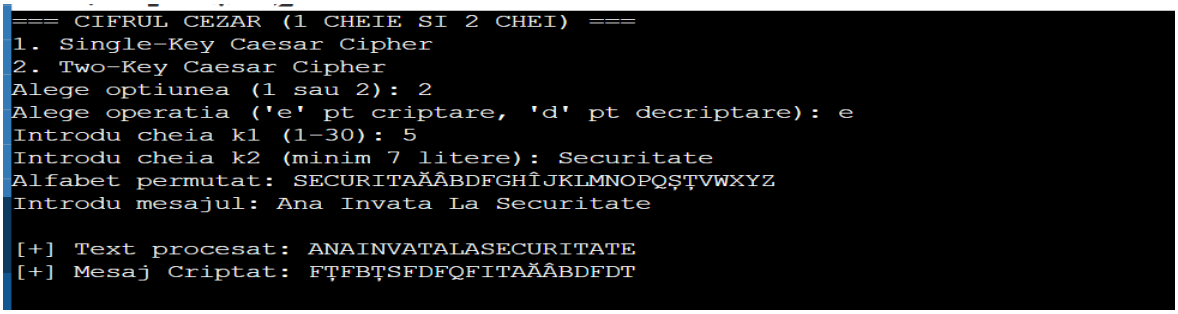


Figura 3. Criptare folosind două chei (k1 = 5, k2 = Securitate)



Programul solicită utilizatorului tipul de operație pe care dorește să o efectueze: criptare sau decriptare. Utilizatorului i se cere să introducă cheia cifrului Cezar, care trebuie să fie un număr întreg între 1 și 30 (inclusiv).

În continuare, utilizatorului i se cere să introducă mesajul pe care dorește să îl cripteze sau să îl decripteze. Programul îl va converti în litere majuscule din alfabetul românesc (31 de litere) și elimină spațiile din datele introduse.

Apoi programul efectuează operația selectată (criptare sau decriptare) folosind cheia și mesajul furnizate. În exemplul dat, se criptează mesajul de la tastatură folosind cheia k1 și cheia k2 ("Securitate"), rezultând alfabetul permutat și mesajul criptat corespunzător.

În final, programul afișează rezultatul, fie mesajul criptat, fie cel decriptat, în funcție de operația selectată.

Cod: https://github.com/gsohptgs/Criptografie-Lab1/tree/main

CONCLUZIE:

În această lucrare de laborator am implementat și analizat algoritmul Cifrul Cezar în limbajul Python, adaptat pentru alfabetul limbii române (31 de litere). Am realizat două variante ale algoritmului: varianta simplă cu o singură cheie (k1) și varianta extinsă cu două chei (k1 și k2). În urma testării, am observat că Cifrul Cezar simplu oferă un nivel scăzut de securitate din cauza spațiului redus de căutare (doar 30 de chei posibile), putând fi spart ușor prin atacul cu forță brută. În schimb, utilizarea celei de-a doua chei k2 pentru permutarea alfabetului sporește rezistența criptografică a algoritmului prin extinderea combinațiilor posibile.

BIBLIOGRAFIE

1. **GEEKSFORGEEKS:** Cryptography and its Types. [accesat 06.09.2023].

Link de acces:

https://www.geeksforgeeks.org/computer-networks/cryptography-and-its-types/

https://www.geeksforgeeks.org/computer-networks/cryptography-and-its-types/

2. **PYTHON DOCUMENTATION:** String Methods & Data Structures. [accesat 15.09.2026].

Link de acces:

https://docs.python.org/3/library/stdtypes.html](https://docs.python.org/3/library/stdtypes.html

ANEXA A: Codul fișierului main.py

```
# Cifrul Cezar - Implementare UTM FCIM (Sarcina 1.1 si 1.2)
# Alfabetul romanesc extins (31 de litere)
ALFABET = "AĂÂBCDEFGHIÎJKLMNOPQRSȘTȚUVWXYZ"
N = len(ALFABET)

def curata_text(text):
    text_curat = ""
    for char in text.upper():
        if char in ALFABET:
            text_curat += char
    return text_curat

def genereaza_alfabet_permutat(k2_text):
    k2_curat = curata_text(k2_text)
    cheie_unica = ""
    for char in k2_curat:
        if char not in cheie_unica:
            cheie_unica += char

    restul = ""
    for char in ALFABET:
        if char not in cheie_unica:
            restul += char

    return cheie_unica + restul

def criptare(text, k1, alfabet):
    rez = ""
    for char in text:
        idx = alfabet.index(char)
        nou_idx = (idx + k1) % N
        rez += alfabet[nou_idx]
    return rez

def decriptare(criptograma, k1, alfabet):
    rez = ""
    for char in criptograma:
        idx = alfabet.index(char)
        nou_idx = (idx - k1) % N
        rez += alfabet[nou_idx]
    return rez

def main():
    print("=== CIFRUL CEZAR (1 CHEIE SI 2 CHEI) ===")
    print("1. Single-Key Caesar Cipher")
    print("2. Two-Key Caesar Cipher")

    opt = input("Alege optiunea (1 sau 2): ").strip()
    op = input("Alege operatia ('e' pt criptare, 'd' pt decriptare):
").strip().lower()

    k1 = int(input(f"Introdu cheia k1 (1-{N-1}): "))

    alfabet_lucru = ALFABET
    if opt == '2':
        k2 = input("Introdu cheia k2 (minim 7 litere): ").strip()
        alfabet_lucru = genereaza_alfabet_permutat(k2)
        print("Alfabet permutat:", alfabet_lucru)

    msg = input("Introdu mesajul: ").strip()
    msg_curat = curata_text(msg)

    if op.startswith('e'):
```

```
            rez = criptare(msg_curat, k1, alfabet_lucru)
            print("\n[+] Text procesat:", msg_curat)
            print("[+] Mesaj Criptat:", rez)
        else:
            rez = decriptare(msg_curat, k1, alfabet_lucru)
            print("\n[+] Criptograma procesata:", msg_curat)
            print("[+] Mesaj Decriptat:", rez)

if __name__ == "__main__":
    main()
```